

Programming Languages

7. Syntax (구문)
(Chapter 6)

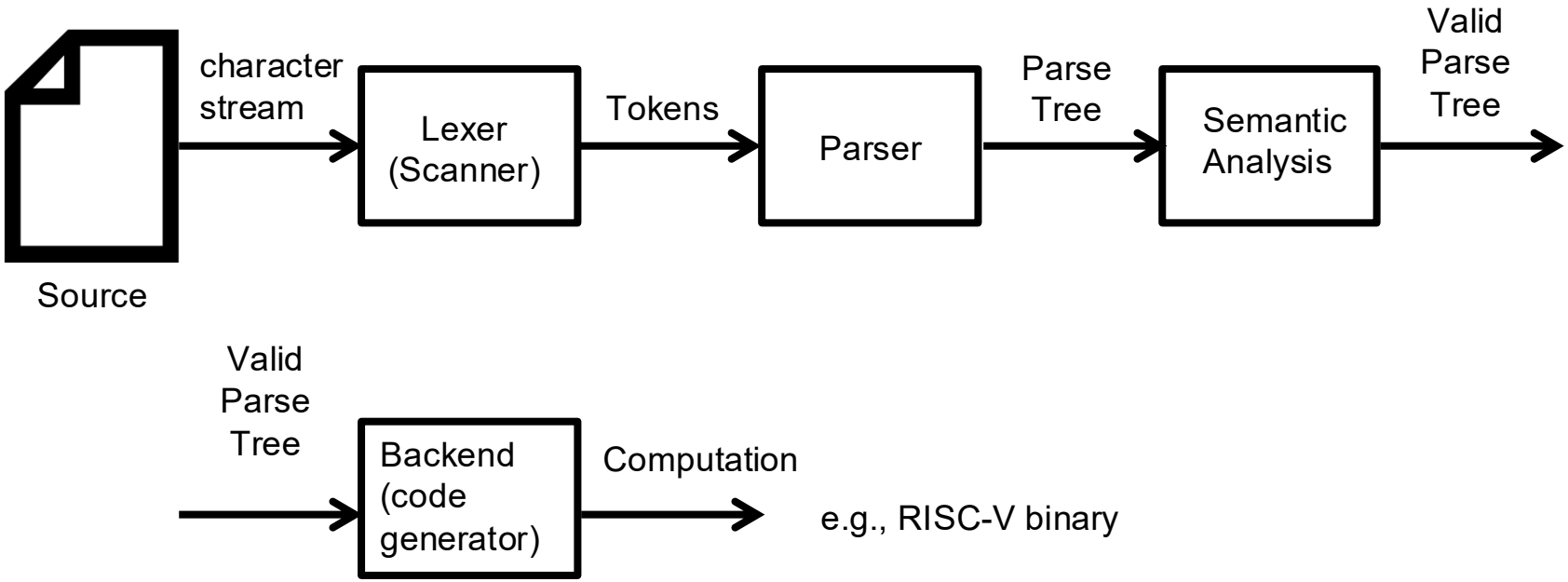
Ji Eun Kim

Fall 2026

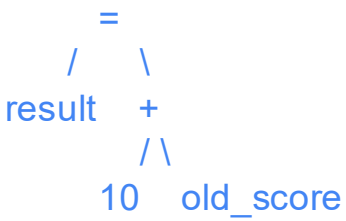
Topics

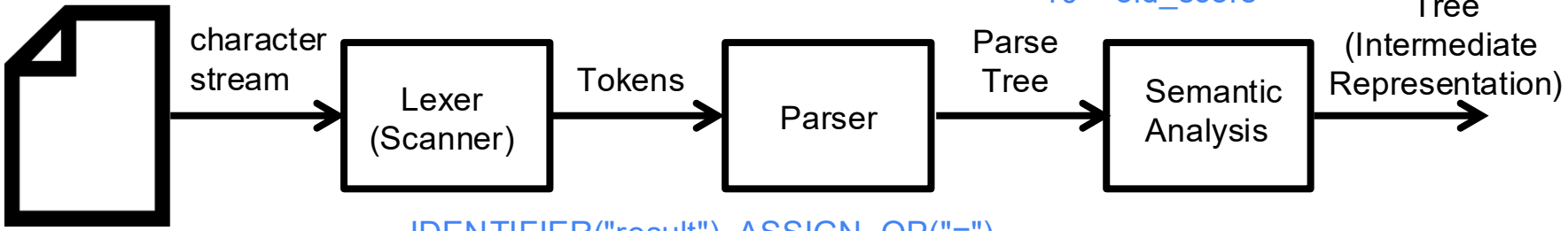
- Lexical Analysis/Scanning
  - Regular Expression
- Formal Description
  - CGF (Context-Free Grammar)
  - BNF (Backus-Naur Form)
  - EBNF (Extended BNF))
  - Syntax Graph
- Syntactic Analysis/Parsing
  - Parse Tree
  - Abstract Syntax Tree
  - Ambiguity(모호성), Precedence(우선순위), Associative (결합규칙)

[Recap] Overview of Program Interpretation

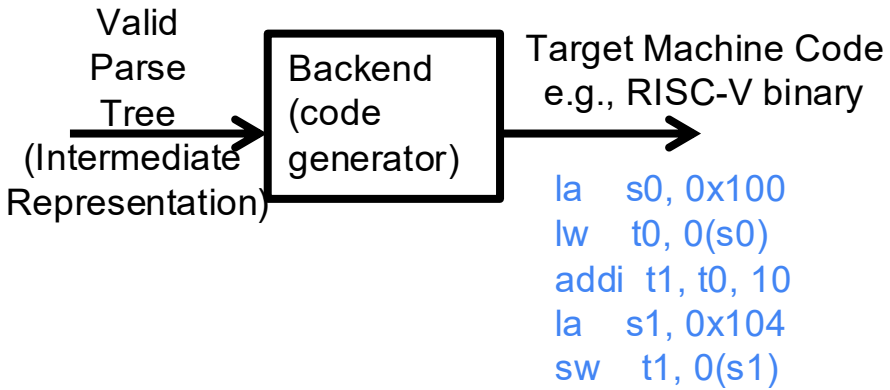


Overview of Program Interpretation





- Have result and old_score been declared?
- Are 10 and old_score types that can be added together (e.g., both are numbers)?
- Can the result of the addition be assigned to the result variable?



Syntax and Semantics

- **Defining** a programming language:
  - Specifications of syntax
    - **Syntax** – **structure** (form) of programs
      - the form a program in the language must take.
  - Specifications of semantics
    - **Semantics** - the **meaning** of programs

- Precise definition, without ambiguity
  - Given a program, there is **only one unique interpretation**.

Scanning and Parsing

- **Lexical Structure (어휘 구조):** The structure of **tokens** (words)
  - Scanning phase (lexical analysis): scanner/lexer
  - Recognizes tokens from characters
- **Syntactical Structure (구문 구조):** The structure of **programs**
  - Parsing phase (syntax analysis): parser
  - Determines the syntactic structure



Purpose of Describing Syntax and Semantics

- Purpose for
  - language designers: Convey the design principles of the language
  - language implementers: Define precisely what to be implemented
  - language programmers: Describe the language that is to be used

- How to describe?
  - Natural language: ambiguous
  - Formal ways: especially for syntax

Tokens

Tokens (words): Building blocks of programs

- **Reserved words** (**keywords**): e.g.,

  `if, while, int, return`

- **Literals/constants**: e.g.,
  - numeric literal: `42`
  - string literal: `"hello"`
- **Special symbols**: e.g., “`;`”, “`<=`”, “`+`”
- **Identifiers**: e.g., `x24, monthly_balance, putchar`

Reserved words vs. Predefined identifiers

- **Reserved words (예약어):**
  - cannot be redefined.
    - e.g., `double if;` is illegal.

- **Predefined identifiers (미리 정의된 식별자):**
  - have initial meaning
  - allow redefinition (not a good idea in practice)
    - e.g., `String`,`Object`,`System`,`Integer` in Java

Principle of Longest Substring (최장 부분문자열 원칙)

- `doif` vs. `do if`
- `x12` vs. `x 12`
- The longest possible string of characters is collected into a **single token.**
- An exception: FORTRAN (공백 무시)
  - `DO 99 I = 1.10` (the same as `DO99I=1.10`)

White Space

- Principle of **longest substring** requires that tokens are separated by **white space**.
- **White space** (token delimiters):
  - blanks, newlines, tabs
  - Ignored except that they separate tokens
- **Free-format language:** format has no effect on the program structure (other than the principle of the longest string)
  - Most languages are free format (e.g., Java, C, ..)
  - an exception: python

Indentation in Python

```
    def perm(l):                          #error: first line indented
 for i in range(len(l)):                  #error: not indented
    s = l[:i] + l[i+1:]
      p = perm(l[:i] + l[i+1:])           #error: unexpected indent
      for x in p:
           r.append(l[i:i+1] + x)
         return r                         #error: inconsistent indent
```

```
def perm(l):
    for i in range(len(l)):
          s = l[:i] + l[i+1:]
          p = perm(l[:i] + l[i+1:])
          for x in p:
              r.append(l[i:i+1] + x)
    return r
```

Regular Expression

- A form for representing sets of strings
- Description of **patterns of characters**
- Basic operations:
  - **Concatenation** done by sequencing the items
  - **Repetition** *
  - **Selection** |
- **[ ]** with a hyphen indicate a range of characters
- **?** indicates an **optional** item
- **Period** indicates any character

Regular Expression

Name	**RE (Regular Expression)**
epsilon (empty set of strings)	ε
symbol	a
concatenation	AB
selection	A | B
repetition	A*

- Shortcuts: **A+** = AA^* **A?** = $A|\varepsilon$ **[a-z]** = (a|b|...|z)

`[a-z][a-z0-9]*`

반드시 소문자로 시작
소문자나 문자는 반복 사용

`(a|b)*aa(a|b)*`

`[0-9]+(\.[0-9]+)?`

정수가 한번이상
소숫점이 있거나 없는 것 허용

More info: https://www3.ntu.edu.sg/home/ehchua/programming/howto/Regexe.html#zz-1.6

Regular Expressions

Examples:

- `(a|b)*c`
  - `matches: abababaac, aac, babbc, c`
  - `non-matches: aaaab,  bca`
- Integer constants of **one or more digits**: `[0-9]+`
  - `vs. 0 or more digits: [0-9]*`
- Unsigned floating-point literals: `[0-9]+(\.[0-9]+)?`
  - `matches: 123, 123.45`
- Most modern text editors use regular expressions in text searches
- Unix utilities such as ***lex/flex:*** generate lexical analyzer/scanner automatically
  - a regular expression description of the tokes of a language -> lex -> lex analyzer

Exercise: Regular Expression (1)

- Describe the strings that are represented by the regular expression:

```
[0-9]+((E|e)(\+|\-)?[0-9]+)?
```

- Examples?



Exercise: Regular Expression (2)

- Write a regular expression for **C-like identifiers** consisting of *letters*, *digits*, and the underscore character '_', and *starting with a letter or underscore*.
- Hint: '_' can be used as an element of range e.g. [a-z_]
- 

- **Match:** `count`, `_is_valid`, `temp1`, `user_name`
- **Non-match:** `1st_place`, `$amount`, `my-variable`, `user name`

Exercise: Regular Expression (3)

- 사용자명@도메인.최상위도메인' 형식의 간단한 이메일 주소
  - 단 문자와 숫자만 사용
- Hint: use () and \ , 최소 2번이상 반복: {2,}
-  {2,}

- **Match:** `test@example.com, user.name@domain.co.kr, dev-1@my-site.net`
- **Non-match:** `test@.com, user@domain, hello world@e.com, test@exampl`

Tasks of a Scanner

- Recognizes keywords
- Recognizes special characters
- Recognizes identifiers, integers, reals, decimals, strings, etc.
- Ignores white spaces and comments



Scanner

- Simple scanner input:

```
* + ( ) 42 # 345
```

- Produces this output:

```
TT_TIMES
TT_PLUS
TT_LPAREN
TT_RPAREN
TT_NUMBER: 42
TT_ERROR: #
TT_NUMBER: 345
TT_EOL
```

```
(1)  #include <ctype.h>
(2)  #include <stdio.h>

(3)  /* the tokens as an enumerated type */
(4)  typedef enum {TT_PLUS, TT_TIMES, TT_PAREN, TT_RPAREN,
(5)                TT_EOL,TT_NUMBER,TT_ERROR} TokenType;

(6)  int numVal;   /* computed numeric value of a NUMBER token */
(7)  int currChar; /* current character */

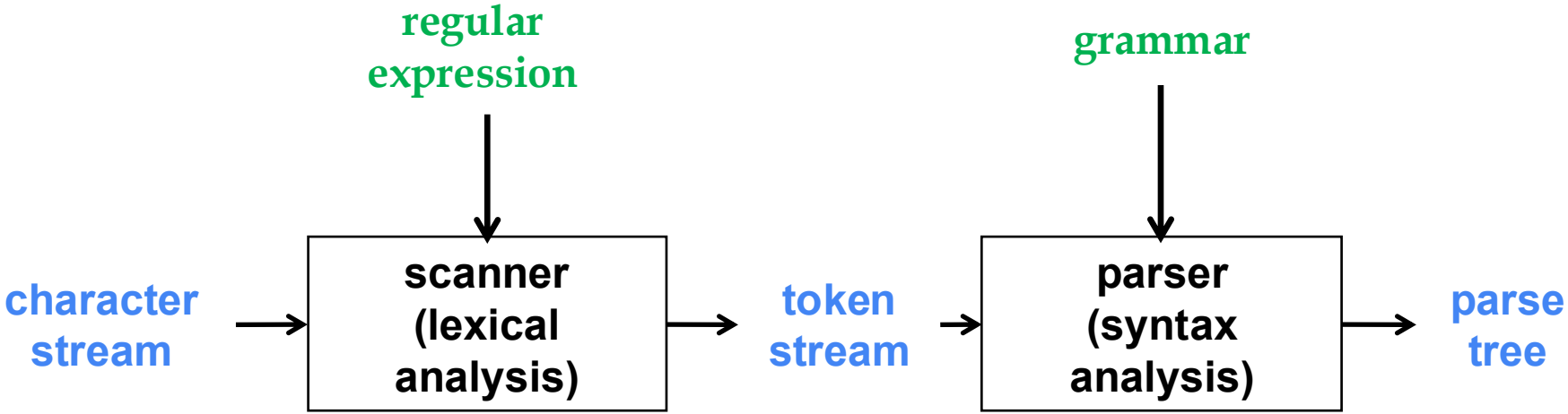
(8)  TokenType getToken(){
(9)      while (currChar == ' ')         /* skip blanks */
(10)         currChar = getchar();
(11)     if (isdigit(currChar)){         /* recognize a NUMBER tc
(12)         numval = 0;
(13)         while (isdigit(currChar)){
(14)             /* compute numeric value */
(15)             numval = 10 * numval + currChar - '0';
(16)             currChar = getchar();
(17)         }
(18)         return TT_NUMBER;
(19)     }
(20)     else{                           /* recognize a special symbol */
(21)         switch (currChar){
(22)             case '(':  return TT_LPAREN; break;
(23)             case ')':  return TT_RPAREN; break;
(24)             case '+':  return TT_PLUS; break;
(25)             case '*':  return TT_TIMES; break;
(26)             case '\n': return TT_EOL; break;
(27)             default:   return TT_ERROR; break;
(28)         }
(29)     }
(30) }
```

```
(31) main(){
(32)     TokenType token;
(33)     currChar = getchar();
(34)     do{
(35)         token = getToken();
(36)         switch (token){
(37)             case TT_PLUS:   printf("TT_PLUS\n"); break;
(38)             case TT_TIMES:  printf("TT_TIMES\n"); break;
(39)             case TT_LPAREN: printf("TT_LPAREN\n"); break;
(40)             case TT_RPAREN: printf("TT_RPAREN\n"); break;
(41)             case TT_EOL:    printf("TT_EOL\n"); break;
(42)             case TT_NUMBER: printf("TT_NUMBER: %d\n", numval); breal
(43)             case TT_ERROR:  printf("TT_ERROR: %c\n", curr_char); brea
(44)         }
(45)     } while (token != TT_EOL);
(46)     return 0;
(47) }
```

igure 6.1 A scanner for simple integer arithmetic expressions

Scanning and Parsing

- **Lexical Structure:** The structure of tokens (words)
- **Syntactical Structure:** The structure of programs



Grammars

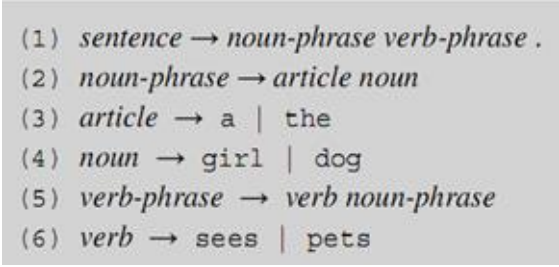
- Example: simple grammar

(1) *sentence* → *noun-phrase verb-phrase* .
(2) *noun-phrase* → *article noun*
(3) *article* → `a` | `the`
(4) *noun* → `girl` | `dog`
(5) *verb-phrase* → *verb noun-phrase*
(6) *verb* → `sees` | `pets`

Figure 6.2 A grammar for simple english sentences

- → separates left and right sides
- | indicates a choice

Grammars



- Language: the programs (character streams) allowed
- Grammar rules (productions): "produce" the language left-hand side, right-hand side
- nonterminals (structured names): *noun-phrase* *verb-phrase*
- terminals (tokens): . `dog` `girl`
- metasymbols: → (“consists of”) | (choice)
- start symbol: the nonterminal that stands for the entire structure (sentence, program).
  - *sentence*
  - *E.g., if-statement* → `if (`*expression*`)` *statement* `else` *statement*

Grammars Produce Languages

- Language: the set of strings (of terminals) that can be generated from the start symbol by **derivation (파생)** (⇒):

sentence
⇒ *noun-phrase verb-phrase* . (rule 1)
⇒ *article noun verb-phrase* . (rule 2)
⇒ `the` *noun verb-phrase* . (rule 3)
⇒ `the girl` *verb-phrase* . (rule 4)
⇒ `the girl` *verb noun-phrase* . (rule 5)
⇒ `the girl sees` *noun-phrase* . (rule 6)
⇒ `the girl sees` *article noun* . (rule 2)
⇒ `the girl sees a` *noun* . (rule 3)
⇒ `the girl sees a dog` . (rule 4)

(1) *sentence* → *noun-phrase verb-phrase* .
(2) *noun-phrase* → *article noun*
(3) *article* → `a` | `the`
(4) *noun* → `girl` | `dog`
(5) *verb-phrase* → *verb noun-phrase*
(6) *verb* → `sees` | `pets`

Figure 6.2 A grammar for simple english sentences

Context-Free Grammar

- Context-Free Grammars (CFG)
  - Noam Chomsky, 1950s.
  - Define *context-free languages.*
  - *Four components:*
    - *terminals, nonterminals, one start symbol, productions (생성규칙)*

What does “Context-Free” mean?

- **Left-hand side** of a production is always **one single nonterminal**:
  - The nonterminal is replaced by the corresponding right-hand side, ***no matter where*** the nonterminal appears.
  - i.e., there is **no context** in such replacement/derivation.

- **Context-sensitive** grammar (context-sensitive languages)
  - e.g. use the verb ”pets” only when the subject is “girl”
  - e.g. articles at the beginning of sentences should be capitalized.

Backus-Naur Form(BNF)

- A meta language used to describe CFG
- John Backus/Peter Naur: for describing the syntax of Algol60.
- BNF is formal and precise
- Notation
  - $\rightarrow$ ::=
  - nonterminal <nonterminal>

Context-Free Grammars and BNFs (cont'd.)

- Rules can express recursion

$$
\begin{aligned}
&expr \rightarrow expr + expr \mid expr * expr \mid (expr) \mid number \\
&number \rightarrow number\ digit \mid digit \\
&digit \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}
$$

Figure 6.4 A simple integer arithmetic expression grammar

$$
\begin{aligned}
number &\Rightarrow number\ digit \\
&\Rightarrow number\ digit\ digit \\
&\Rightarrow digit\ digit\ digit \\
&\Rightarrow 2\ \ digit\ digit \\
&\Rightarrow 23\ \ digit \\
&\Rightarrow 234
\end{aligned}
$$

Figure 6.5 A derivation for the *number* 234 using the grammar of Figure 6.4

Context-Free Grammars and BNFs (cont'd.)

translation-unit → *external-declaration*
| *translation-unit external-declaration*

external-declaration → *function-definition* | *declaration*

function-definition → *declaration-specifiers declarator*
declaration-list compound-statement
| *declaration-specifiers declarator compound-statement*
| *declarator declaration-list compound-statement*
| *declarator compound-statement*

declaration → *declaration-specifiers* ';'
| *declaration-specifiers init-declarator-list* ';'

init-declarator-list → *init-declarator*

함수정의 | 변수 정의

다양한 C언어의 함수 정의

Figure 6.6 Partial BNFs for C (adapted from Kernighan and Ritchie [1988]) (*continues*)

BNF Example: identifier list

- <id_list> ::= <id> | <id>, <id_list>
- <id> ::= ‘a’ | ‘b’ | ‘c’

- What are possible identifier list?



Parse Tree

- Represents the **derivation steps** from start symbol to the string
- Given the derivations used in the parsing of an input sequence, a parse tree has
  - the start symbol as the root
  - the terminals of the input sequence as leafs
  - for each production $A \rightarrow X_1 X_2 \ldots X_n$ used in a derivation, a node A with children $X_1 X_2 \ldots X_n$

Parse Tree Example 1

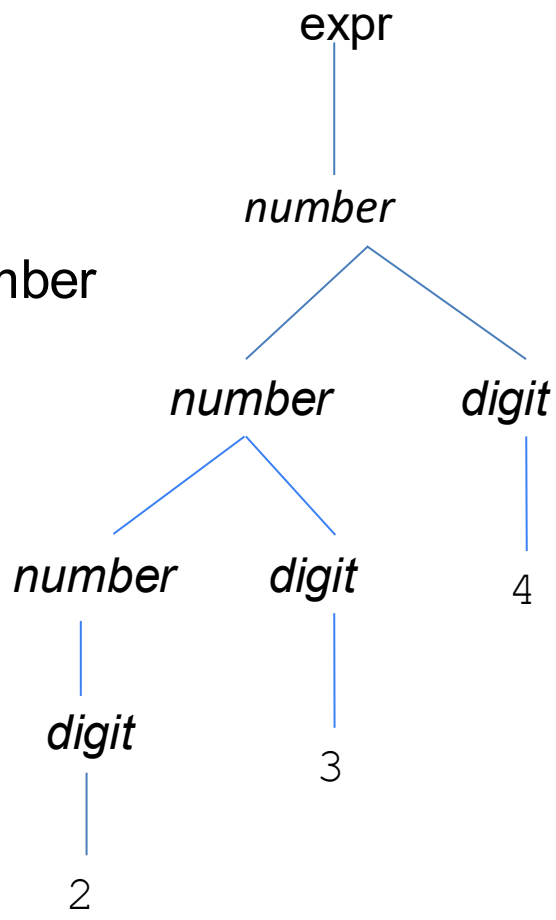
CFG:

expr → expr + expr | expr * expr | (expr) | number

number → number digit |digit

digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Input Sequence: 234



Parse Tree Example 2

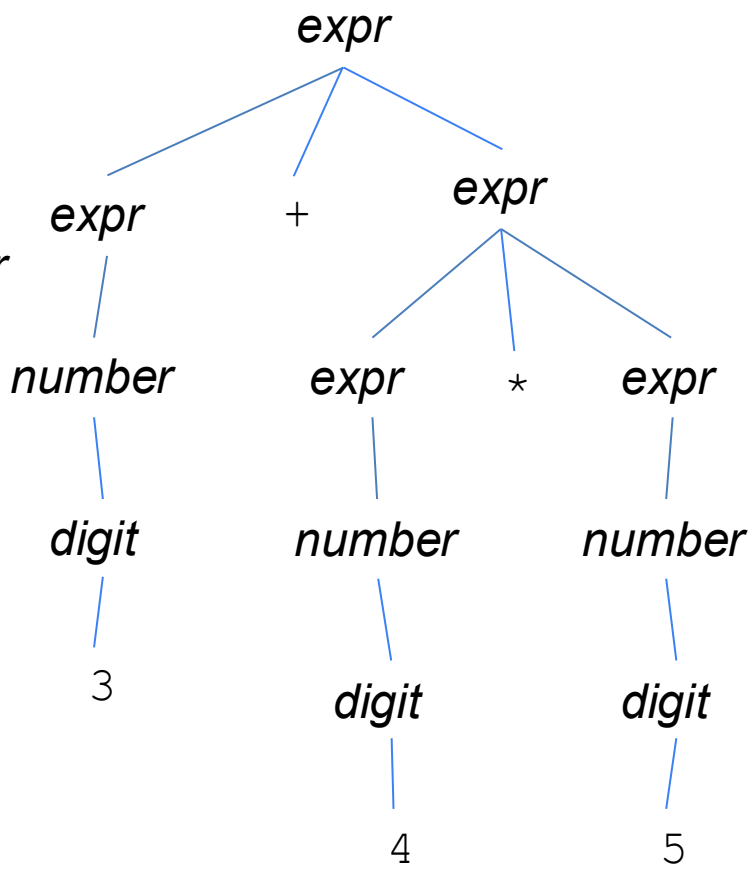
CFG:

expr → *expr* + *expr* | *expr* * *expr* | (*expr*) | *number*

number → *number digit* |*digit*

digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

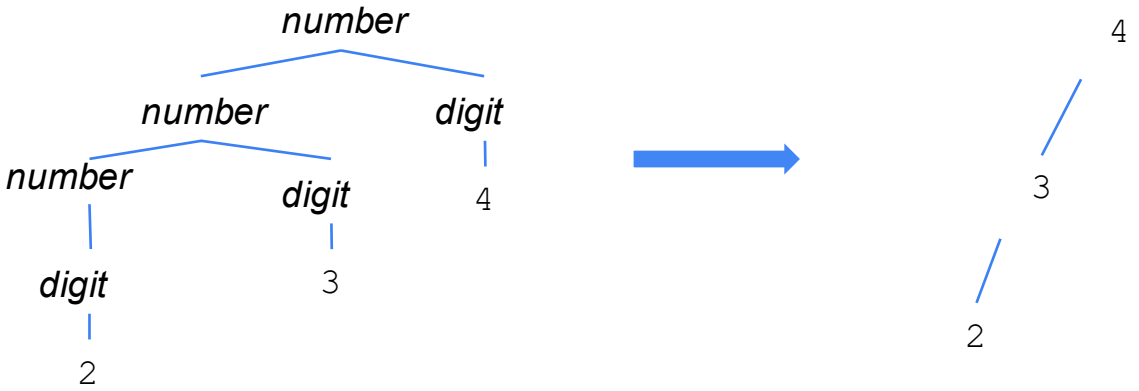
Input Sequence: 3+4*5



* 가. + 보다 우선순위

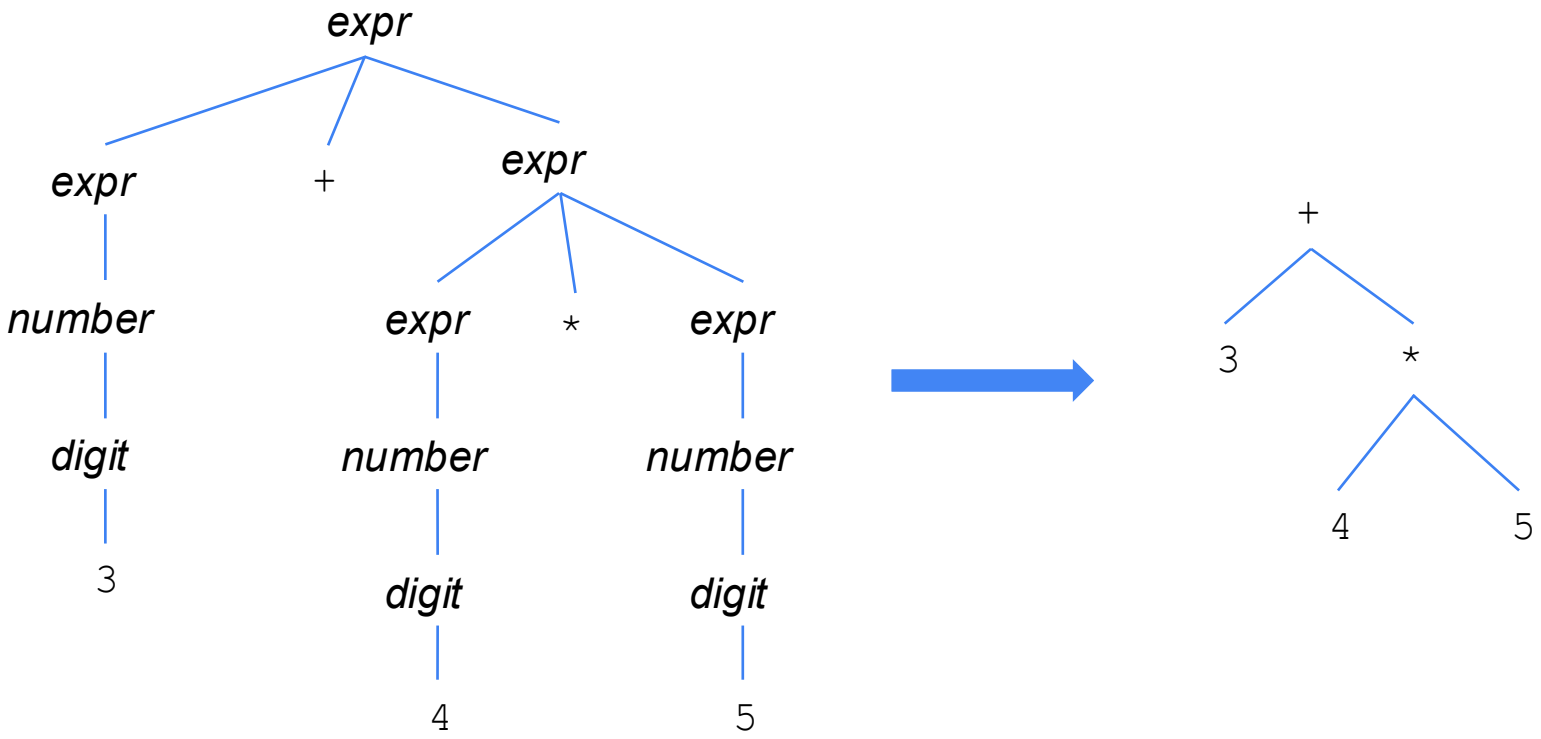
Abstract Syntax Tree

- **Parse Tree**: still tedious, all terminals and nonterminals in a derivation are included in the tree.
- **Abstract Syntax Tree**:
  - Remove ”unnecessary” terminals and nonterminals
  - Still completely determine syntactic structure.

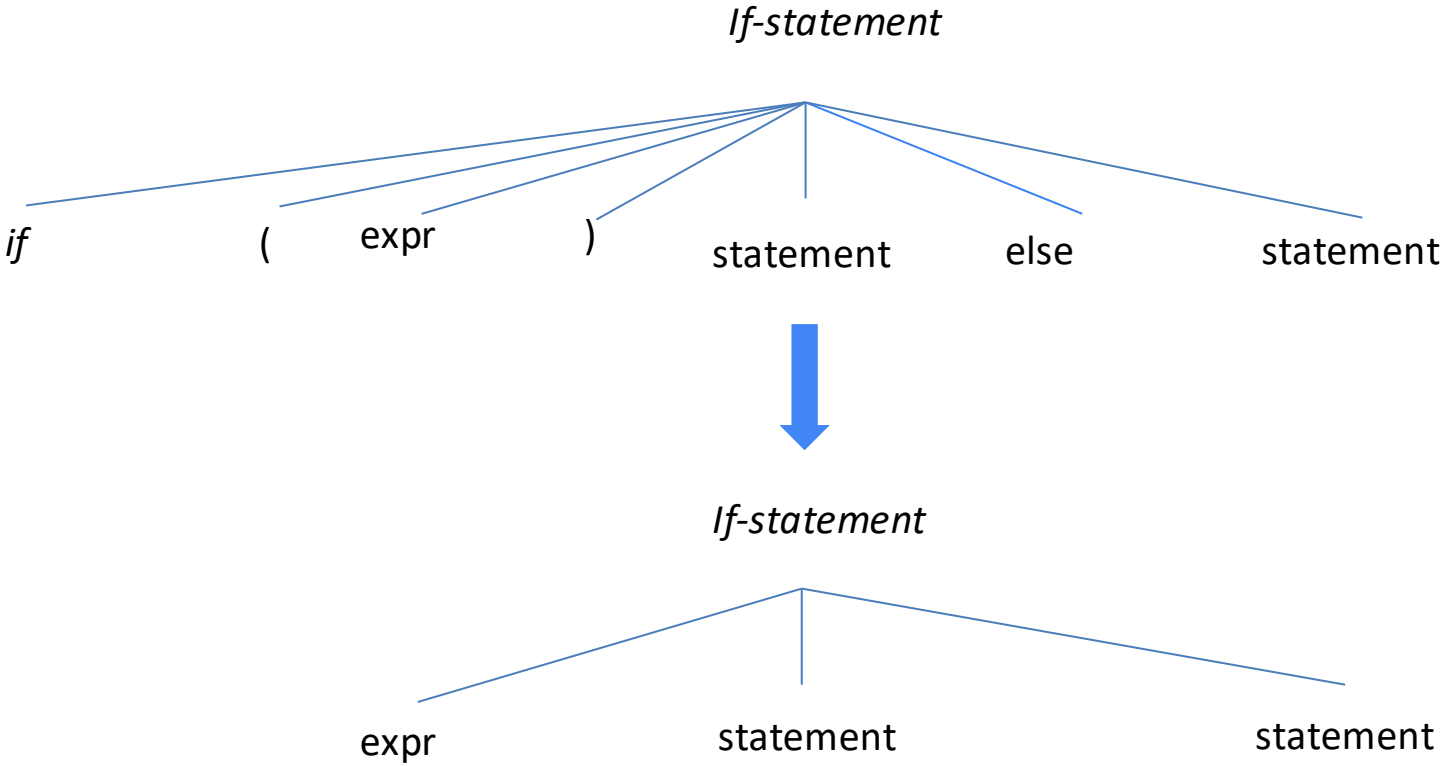


AST Example 1

Input Sequence: 3+4*5



AST Example 2



a bit confusing to the programmer, but important for the language designer and translator

Exercise

draw

1) parse trees
2) abstract syntax trees

```
3 + 4 * 5 + 6 * 7
```

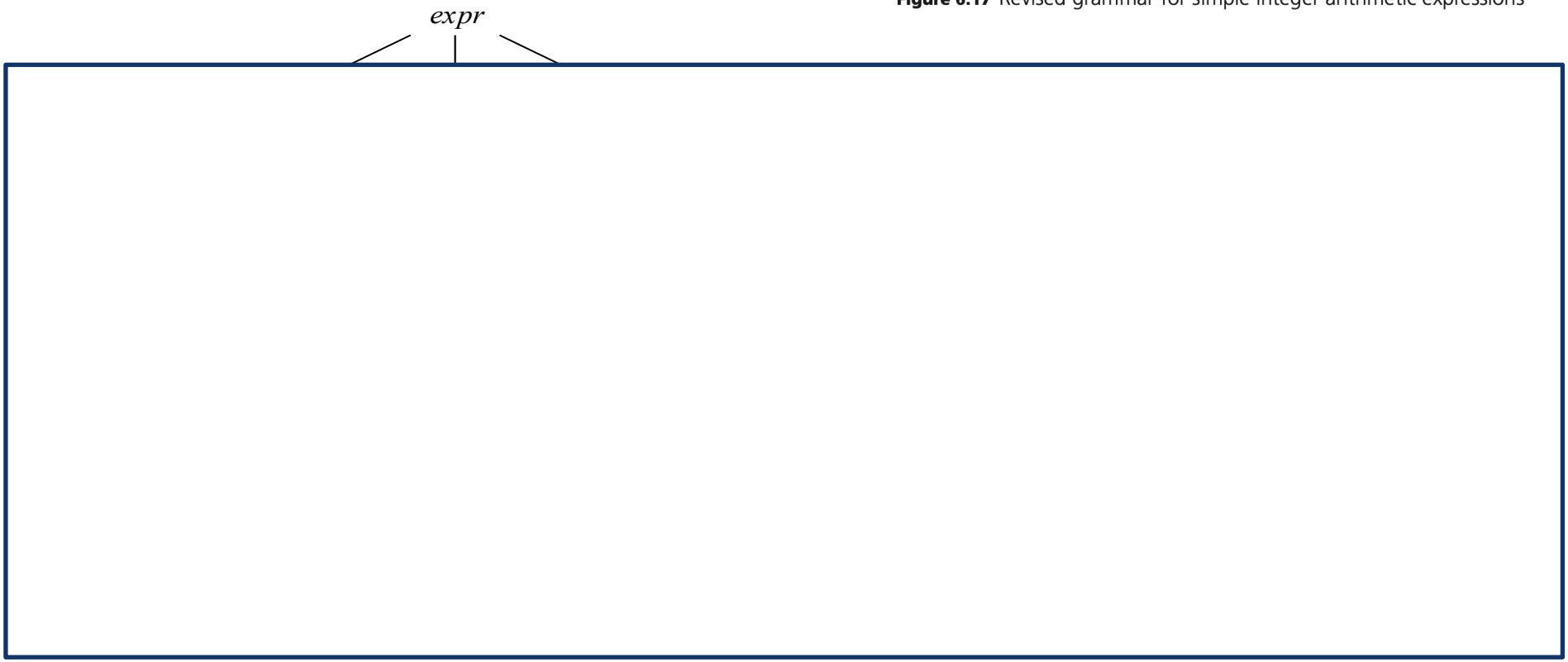
$$
\begin{aligned}
expr &\rightarrow expr \ + \ term \mid term \\
term &\rightarrow term \ * \ factor \mid factor \\
factor &\rightarrow (\ expr \ ) \mid number \\
number &\rightarrow number \ digit \mid digit \\
digit &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}
$$

Figure 6.17 Revised grammar for simple integer arithmetic expressions

3 + 4 * 5 + 6 * 7

$$
\begin{aligned}
expr &\rightarrow expr\ +\ term \mid term \\
term &\rightarrow term\ *\ factor \mid factor \\
factor &\rightarrow (\ expr\ ) \mid number \\
number &\rightarrow number\ digit \mid digit \\
digit &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}
$$

Figure 6.17 Revised grammar for simple integer arithmetic expressions



Exercise

draw

1) parse trees
2) abstract syntax trees

```
3 * 4 + 5 * 6 + 7
```

$$
\begin{aligned}
expr &\rightarrow expr \ + \ term \mid term \\
term &\rightarrow term \ * \ factor \mid factor \\
factor &\rightarrow (\ expr \ ) \mid number \\
number &\rightarrow number \ digit \mid digit \\
digit &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}
$$

Figure 6.17 Revised grammar for simple integer arithmetic expressions

3 * 4 + 5 * 6 + 7

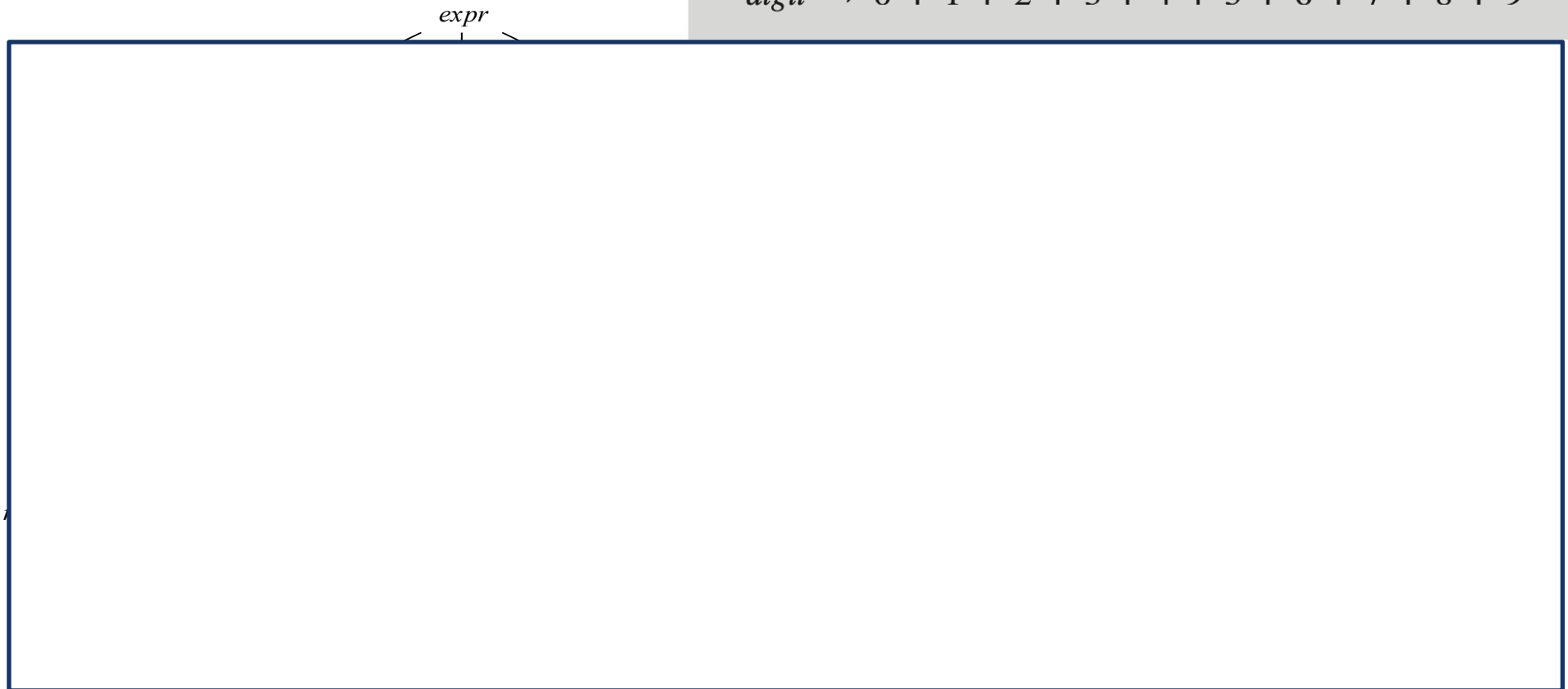
$expr \rightarrow expr + term \mid term$

$term \rightarrow term * factor \mid factor$

$factor \rightarrow (expr) \mid number$

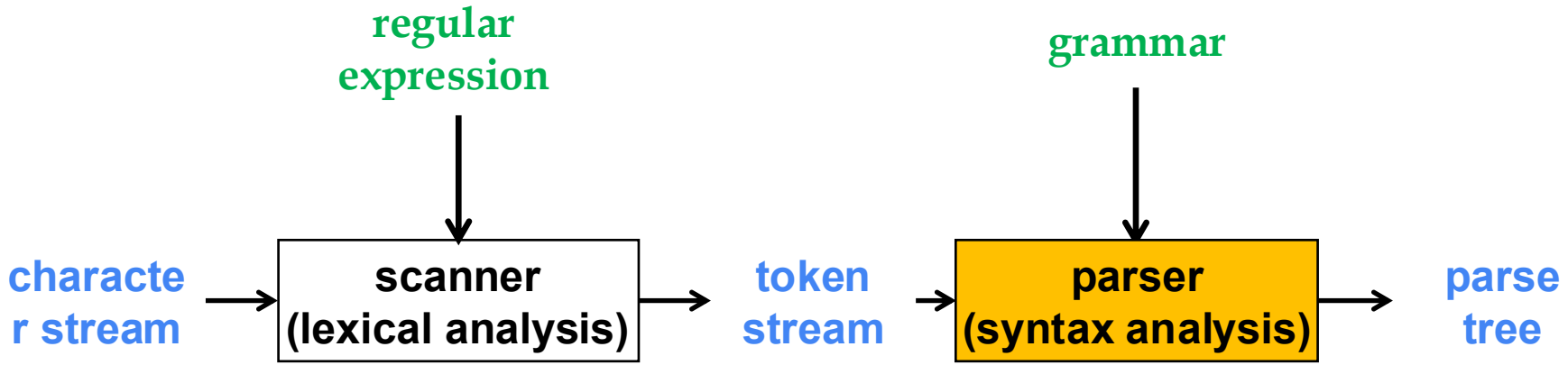
$number \rightarrow number\ digit \mid digit$

$digit \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$



What is Parsing?

- **Given a grammar and a token string:**
  - determine if the grammar can generate the token string?
  - i.e., is the string a **legal** program in the language?
- In other words, to **construct a parse tree for the token string.**



Ambiguity(모호성), Associativity(결합성), and Precedence (우선순위)

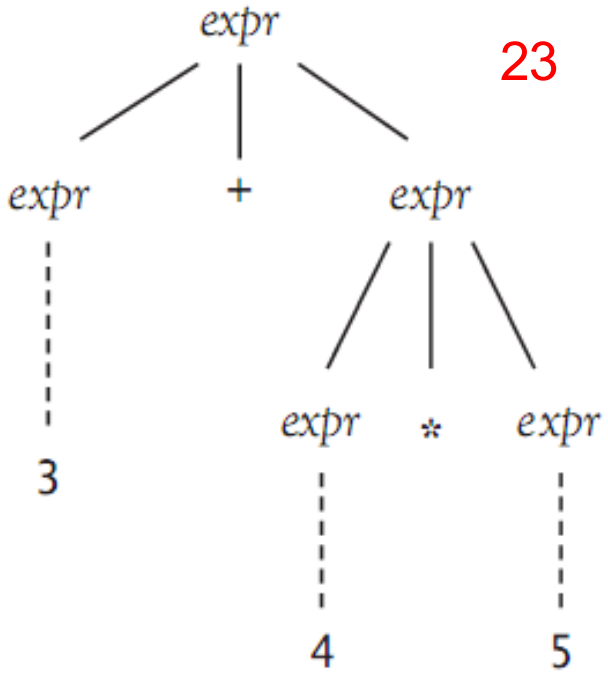
- **A parse tree gives a unique syntactic structure**
- **Leftmost derivation**: the leftmost remaining nonterminal is singled out for replacement at each step
  - Each parse tree has a unique leftmost derivation

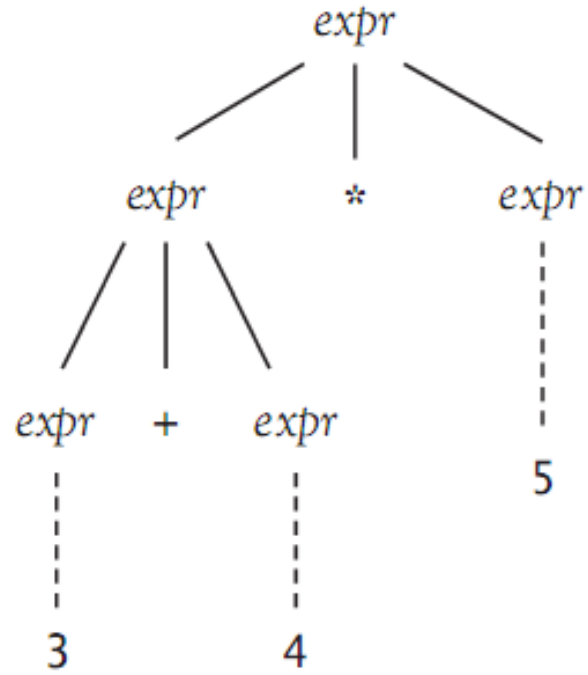
- **Ambiguous grammar:** There can be multiple parse trees for the same sentence (program)
  - In other words, multiple leftmost derivations.

Ambiguity, Associativity, and Precedence (cont'd.)

$expr \rightarrow expr + expr \mid expr * expr \mid (expr) \mid number$
$number \rightarrow number\ digit \mid digit$
$digit \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

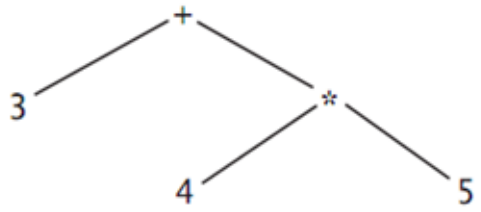
Two different parse trees for expression
3 + 4 * 5



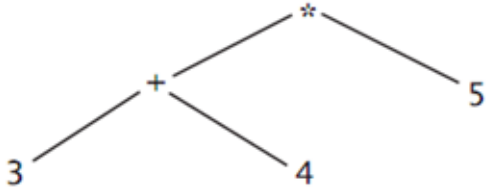


Ambiguity, Associativity, and Precedence (cont'd.)

Tree 1



Tree 2



Leftmost Derivation 1 (Corresponding to Tree 1 of Figure 6.13)	**Leftmost Derivation 2 (Corresponding to Tree 2 of Figure 6.13)**
expr ⇒ *expr* + *expr*	expr ⇒ expr * *expr*
⇒ *number* + *expr*	⇒ *expr* + *expr* * *expr*
⇒ *digit* + *expr*	⇒ *number* + *expr* * *expr*
⇒ 3 + *expr*	⇒ . . . (etc.)
⇒ 3 + *expr* * *expr*	
⇒ 3 + *number* * *expr*	
⇒ . . . (etc.)	

Figure 6.14 Two leftmost derivations for `3 + 4 * 5`, indicating the ambiguity of the grammar of Figure 6.4

Ambiguity, Associativity, and Precedence (cont'd.)

- Ambiguous grammars present difficulties
  - Must either **revise** **them to remove ambiguity** or state a **disambiguating rule**

- Usual way to **revise the grammar** is to write a new grammar rule (called a term) that **establishes a precedence cascade**
  - to force the matching of the "*" at a **lower** **point in the parse tree**

expr → *expr* + *expr* | *term*

term → *term* * *term* | *(expr)* | *term*

*: is shown in the lower point => higher precedence
(낮은 위치에 두어 먼저 계산되게 함)

Ambiguity, Associativity, and Precedence (cont'd.)

- Can replace *expr* → *expr* + *expr*
  - With either
    - *expr* → ***expr*** + *term* **(left-recursive)**
    - *expr* → *term* + ***expr*** **(right-recursive)**
- left-recursive => left associative (왼쪽에서 오른쪽으로 처리)
- right-recursive => right associative

Ambiguity, Associativity, and Precedence (cont'd.)

recursive direction: deeper/lower in the tree -> higher precedence

3+4+5

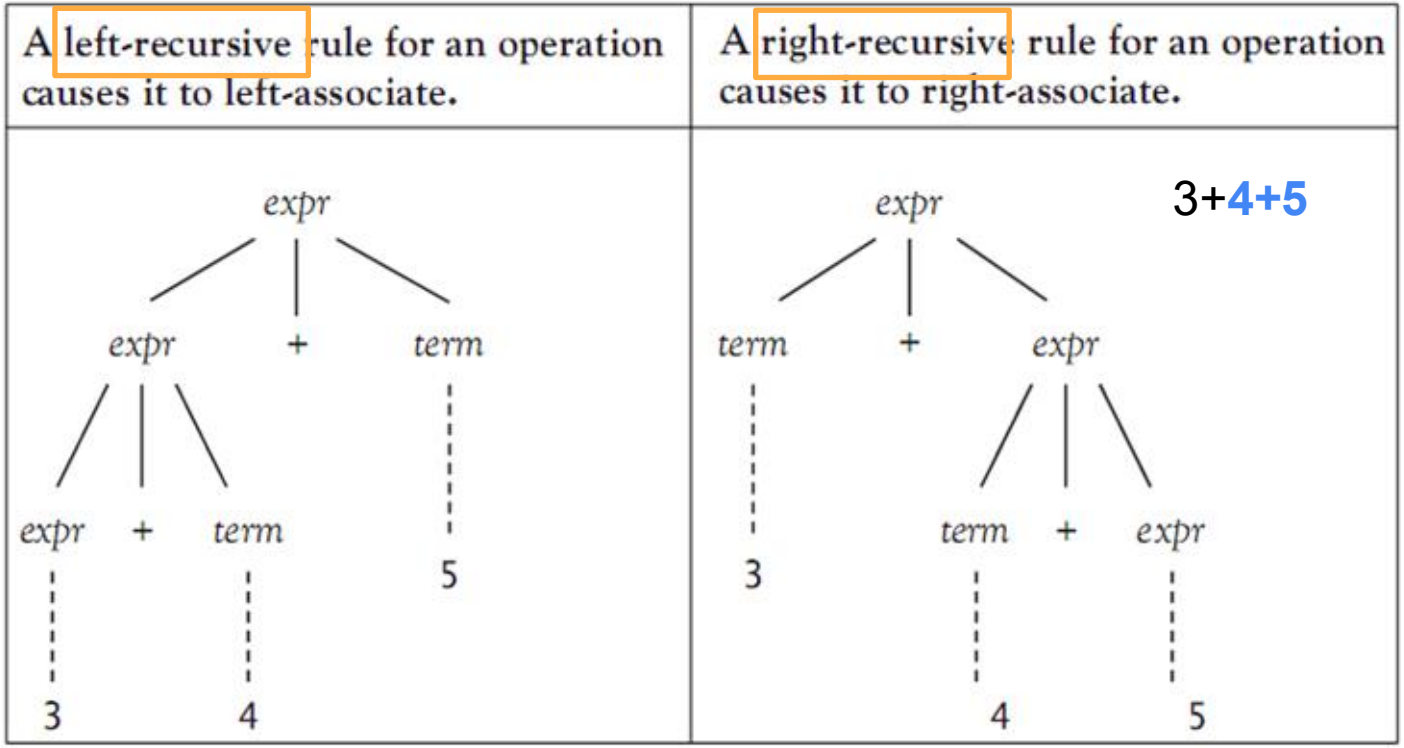


Figure 6.16: Parse trees showing results of left- and right-recursive rules

Ambiguity, Associativity, and Precedence (cont'd.)

- **We want to make + left-associative and * has precedence over +**

$$expr \rightarrow expr \ + \ term \mid term$$ (덧셈은 term들의 결합)

$$term \rightarrow term \ * \ factor \mid factor$$ (곱셈은 factor들의 결합)

$$factor \rightarrow (\ expr \ ) \mid number$$

$$number \rightarrow number \ digit \mid digit$$

$$digit \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

Figure 6.17 Revised grammar for simple integer arithmetic expressions

Note: a proof of this requires more advanced techniques from parsing theory

EBNF (Extended Backus-Naur Form)

- **Extended Backus-Naur** form (or **EBNF**): introduces new notation to handle common issues
- **Repetition { }** : 0 or more repetition

number → *digit* | *number digit* ⇒ *number* → digit { *digit* }

expr → *expr* + *term* | *term* ⇒ *expr* → *term* {+ *term*}

- **Optional part [ ]**

signed-number → *sign number*
| *number* ⇒ *signed-number* → [sign] *number*

if-stmt → `if` (*expr*) *stmt*
| `if` (*expr*) *stmt* `else` *stmt* ⇒ *if-stmt* → `if` (*expr*) *stmt* [`else` *stmt*]

Repetition Explanation

$$number \rightarrow number\ digit \mid digit$$

$$\begin{aligned} number &\Rightarrow number\ digit \\ &\Rightarrow number\ digit\ digit \\ &\Rightarrow number\ digit\ digit\ digit \\ &\Rightarrow digit\ \ldots\ digit \\ &\ldots \\ &\Rightarrow \text{(arbitrary repetitions of digit)} \end{aligned}$$

$$expr \rightarrow expr + term \mid term$$

$$\begin{aligned} expr &\Rightarrow expr + term \\ &\Rightarrow expr + term + term \\ &\Rightarrow expr + term + term + term \\ &\ldots \\ &\Rightarrow term + \ldots + term \end{aligned}$$

EBNF: $number \rightarrow$ digit { $digit$ }

$expr \rightarrow term \{+\ term\}$

Integer Arithmetic Expression (BFN -> EBNF)

BNF

$$
\begin{aligned}
&expr \rightarrow expr \ + \ term \ \mid \ term \\
&term \rightarrow term \ * \ factor \ \mid \ factor \\
&factor \rightarrow (\ expr \ ) \ \mid \ number \\
&number \rightarrow number \ digit \mid digit \\
&digit \ \rightarrow \ 0 \ \mid \ 1 \ \mid \ 2 \ \mid \ 3 \ \mid \ 4 \ \mid \ 5 \ \mid \ 6 \ \mid \ 7 \ \mid \ 8 \ \mid \ 9
\end{aligned}
$$



EBNF

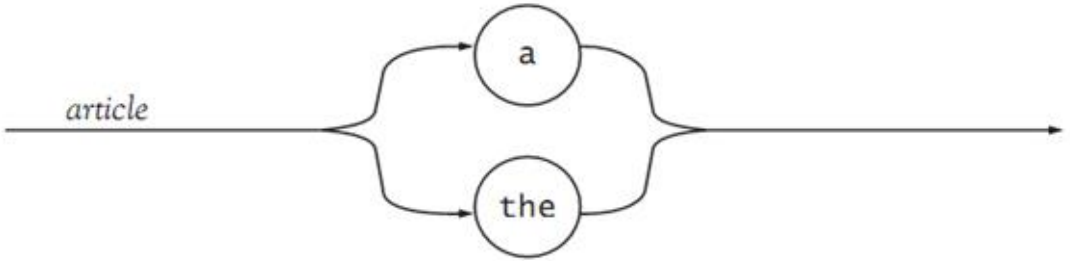
$$
\begin{aligned}
&expr \rightarrow term \ \{ \ + \ term \ \} \\
&term \ \rightarrow \ factor \ \{ \ * \ factor \ \} \\
&factor \ \rightarrow \ (\ expr \ ) \ \mid \ number \\
&number \ \rightarrow \ digit \ \{ \ digit \ \} \\
&digit \ \rightarrow \ 0 \ \mid \ 1 \ \mid \ 2 \ \mid \ 3 \ \mid \ 4 \ \mid \ 5 \ \mid \ 6 \ \mid \ 7 \ \mid \ 8 \ \mid \ 9
\end{aligned}
$$

Syntax Diagrams

- **Syntax diagram**: indicates the sequence of terminals and nonterminals encountered in the right-hand side of the rule
- **circles or ovals** for **terminals**
- **squares** or **rectangles** for **nonterminals**
- Based on EBNF



noun-phrase→ *article* nound



article→ a| the

Figure 6.19: Syntax diagrams for *noun-phrase* and *article* of the simple English grammar presented in Section 6.2

EBNFs and Syntax Diagrams (cont'd.)

- Can condense several rules into one diagram
- Use loops to indicate repetition

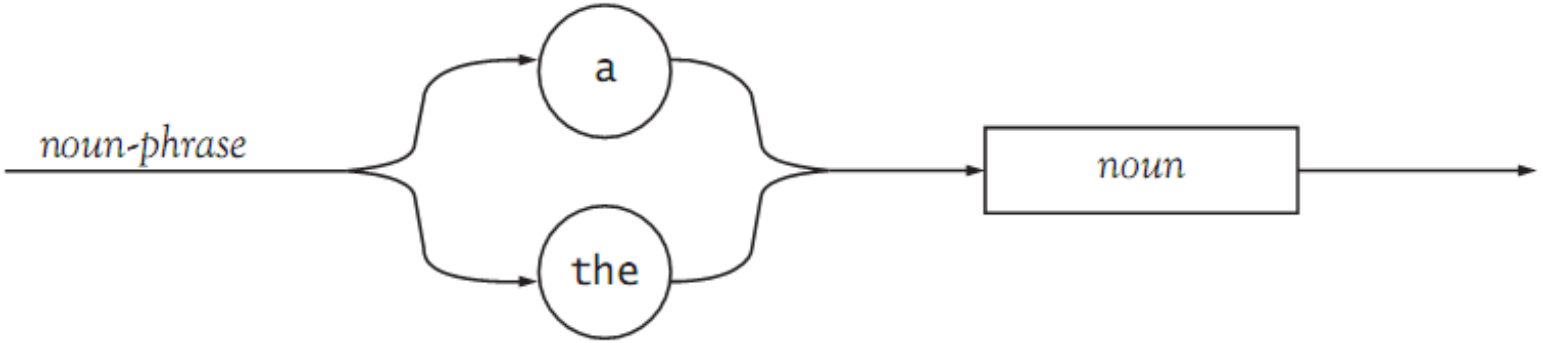
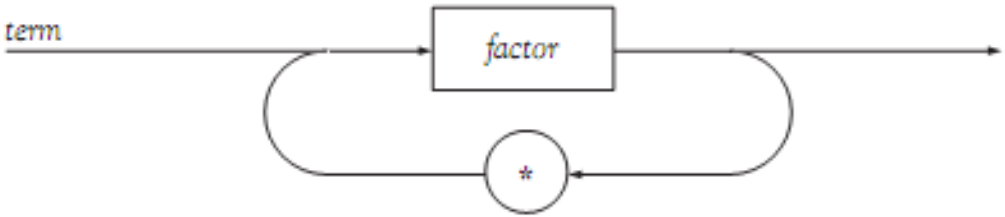


Figure 6.20: Condensed version of diagrams shown in Figure 6.19

EBNFs and Syntax Diagrams (cont'd.)

term→ factor { * factor }



factor → (expr) | number}

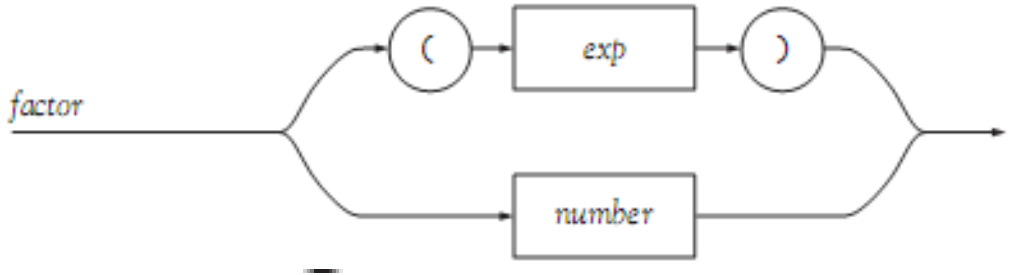
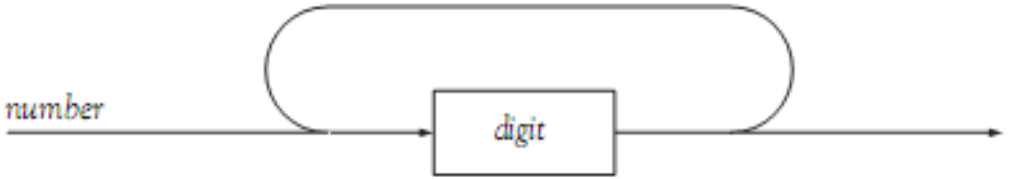
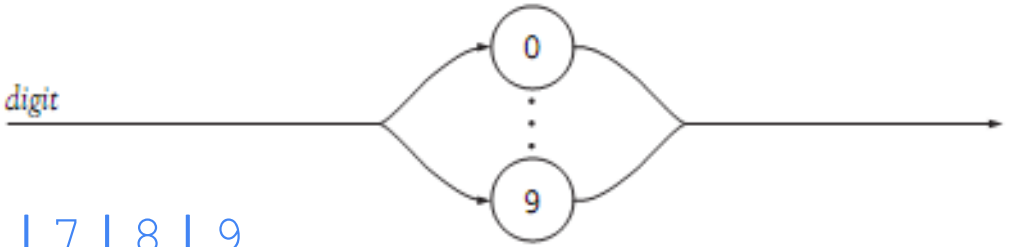


Figure 6.21: Syntax diagrams for a simple integer expression grammar

EBNFs and Syntax Diagrams (cont'd.)

number→ digit {digit}





digit→ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

EBNFs and Syntax Diagrams (cont'd.)

if-stmt → `if` (*expr*) *stmt* [`else` *stmt*]

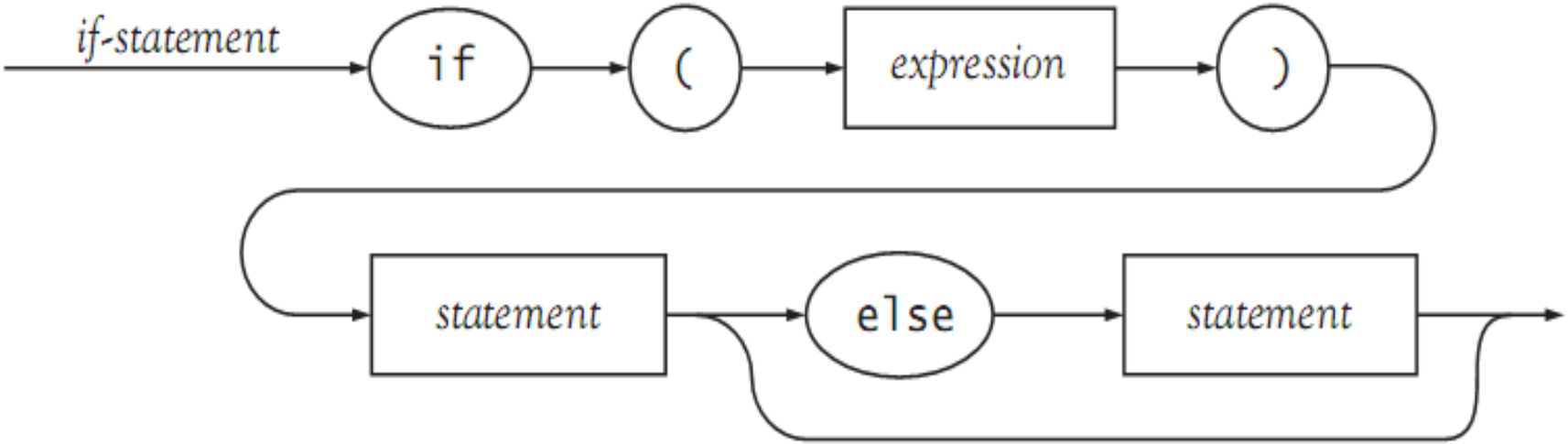


Figure 6.22 Syntax diagram for the *if-statement* in C

Parsing Techniques

- **Parsing:**
  - Determine if a grammar can generate a given token string.
  - That is, to construct a parse tree for the token string.

- **Two ways of constructing the parse tree**
  - Top-down (from root towards leaves)
    - Can be constructed more easily **by hand**
    - expands *nonterminals* to match incoming tokens and directly construct a derivation
  - Bottom-up (from leaves towards root)
    - Can handle a larger class of grammars
    - **Parser generators** tend to use bottom-up methods
    - YACC/Bison generates a parser from the grammar spec. (Compiler Compilers)

References

[1] Programming languages principles and practice, 3rd Edition, Couden & Lambert (Textbook)